TECNICATURA UNIVERSITARIA
EN PROGRAMACIÓN
A DISTANCIA

Trabajo Práctico Integrador
Bases de Datos II

Food Store

Alumnos:

- Gustavo Yoel Santarelli
- Rocio Laura Garay

Grupo: 321

Materia: Bases de Datos II
Año: 2026

Link de GitHub:Repositorio de GitHub

TECNICATURA UNIVERSITARIA
EN PROGRAMACIÓN
A DISTANCIA

1. Descripción

Optamos por realizar este trabajo una plataforma "Food Store", integral de comercio electrónico gastronómico diseñada para optimizar la gestión de pedidos en línea, el catálogo dinámico de productos alimenticios y la administración centralizada de clientes. El sistema aborda una problemática real del sector de la industria de alimentos y bebidas (*Food Delivery & E-commerce*): la necesidad de procesar transacciones concurrentes de alta velocidad, mantener catálogos flexibles con atributos variables según la categoría culinaria (hamburguesas, pizzas, bebidas, etc.) y conservar un historial de auditoría estricto mediante políticas de baja lógica que impiden la pérdida accidental o maliciosa de información histórica.

La arquitectura operativa del negocio se divide en tres actores principales: el Cliente (quien interactúa con la interfaz web para buscar productos, armar su carrito y confirmar pedidos), el Administrador (responsable de actualizar precios, altas de nuevos platos y control de stock) y el Sistema Backend (encargado de persistir los datos de forma segura en la nube mediante MongoDB Atlas, garantizando la consistencia transaccional y la disponibilidad ante picos de demanda).

2. Modelado de Datos (Esquemas y Colecciones)

El diseño consta de 3 colecciones principales relacionadas:

- Users: Almacena la información de los usuarios y clientes registrados.
- Products: Contiene el catálogo de comidas, precios y categorías.
- Orders: Registra las transacciones de compra, asociando al cliente (Linking) y conteniendo los ítems del pedido (Embedding).

Ejemplo de esquema en formato JSON:

```
None
{
  "_id": ObjectId("64a7b2c..."),
  "orderNumber": "ORD-2026-001",
  "customerId": ObjectId("64a7a1b..."),
  "items": [
    { "productId": ObjectId("64a79fc..."), "quantity": 2,
"unitPrice": 4500 }
  ],
  "totalAmount": 9000,
  "status": "PENDING",
```

```
  "active": true
}
```

3. Justificación Técnica: Embedding vs Linking

En el diseño se aplicó una estrategia híbrida según los patrones de acceso:

- Embedding: Utilizado en los ítems dentro de una orden (items). Se justifica porque la cantidad de productos por ticket es acotada (1:N) y se leen siempre en conjunto con la orden, garantizando atomicidad y rendimiento sin joins.

- Linking (Referencias por ID): (N:M) Utilizado para relacionar la orden con el cliente (customerId) y los productos del catálogo. Se justifica debido a que los clientes y productos evolucionan de forma independiente y evitan la duplicación masiva de datos en un entorno de escritura intensiva.

4. Operaciones CRUD y Baja Lógica

La aplicación backend implementa las operaciones del ciclo de vida de los datos asegurando el cumplimiento de la política de baja lógica.

- CREATE: Inserción de nuevos documentos con estado activo por defecto (`active: true`).

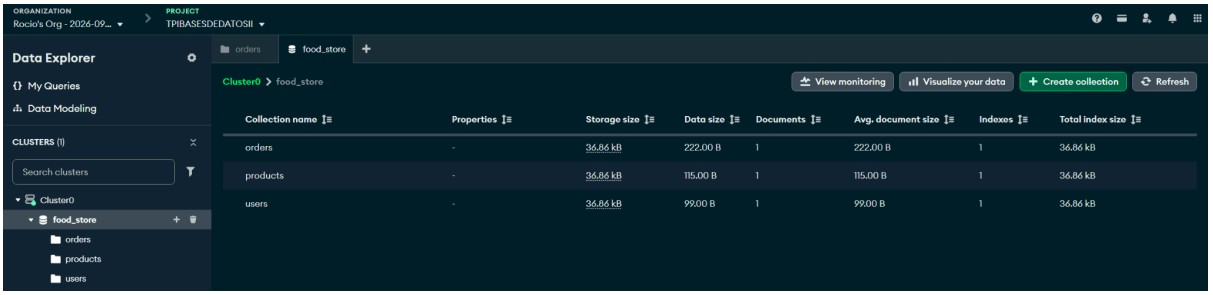
- READ: Consultas filtradas que excluyen registros inactivos (`{ active: true }`).

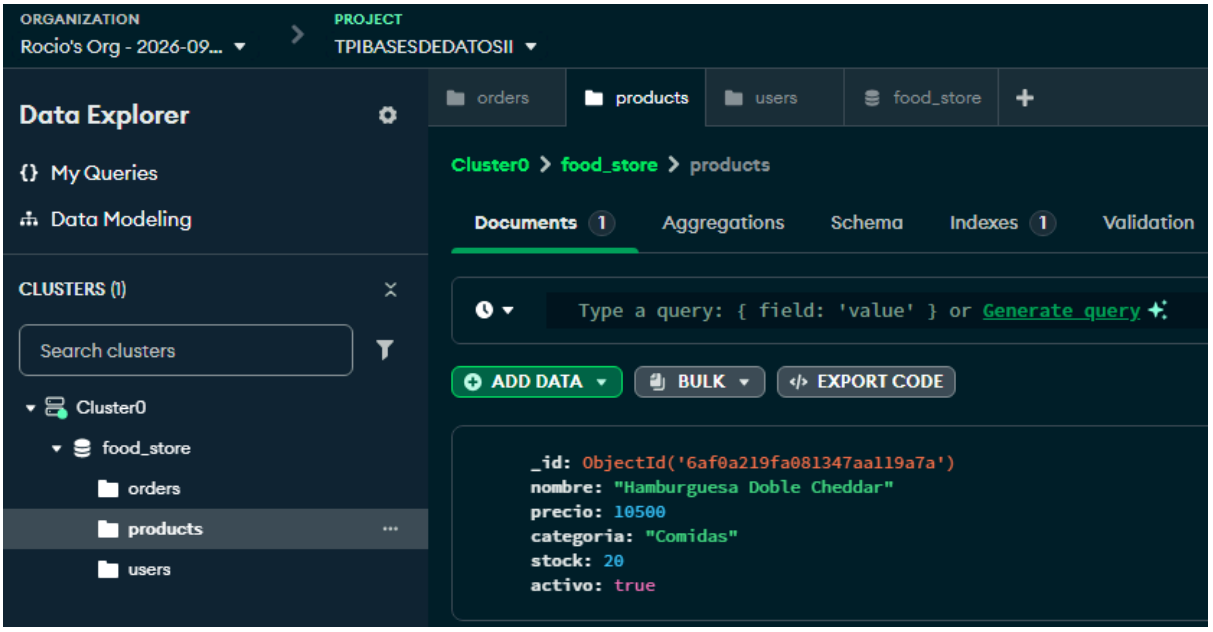
- UPDATE: Modificación de atributos específicos de documentos existentes.

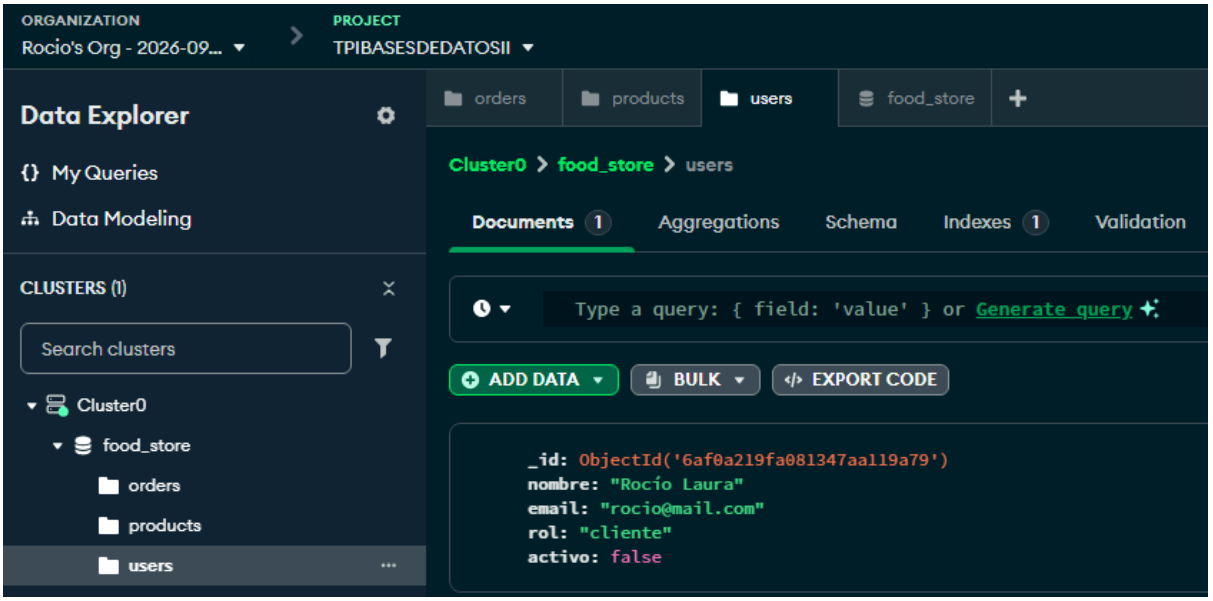
- DELETE (Baja Lógica): Operación `updateOne` que cambia el campo activo de: `true` a `false` a la fecha actual.

5. Capturas del trabajo

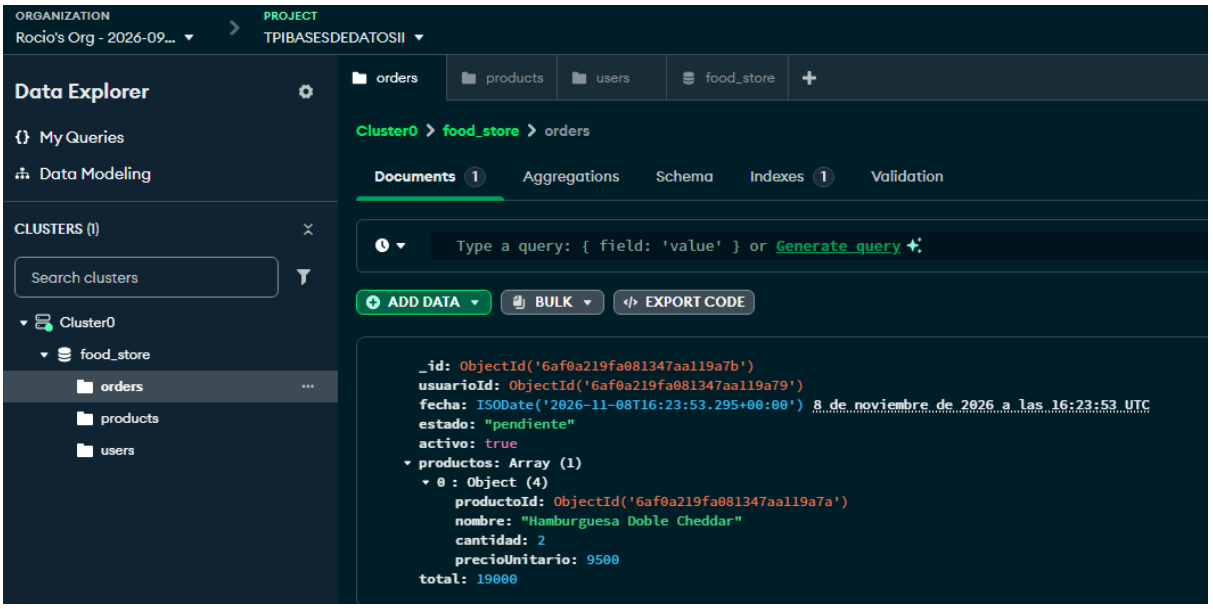
TECNICATURA UNIVERSITARIA
EN PROGRAMACIÓN
A DISTANCIA







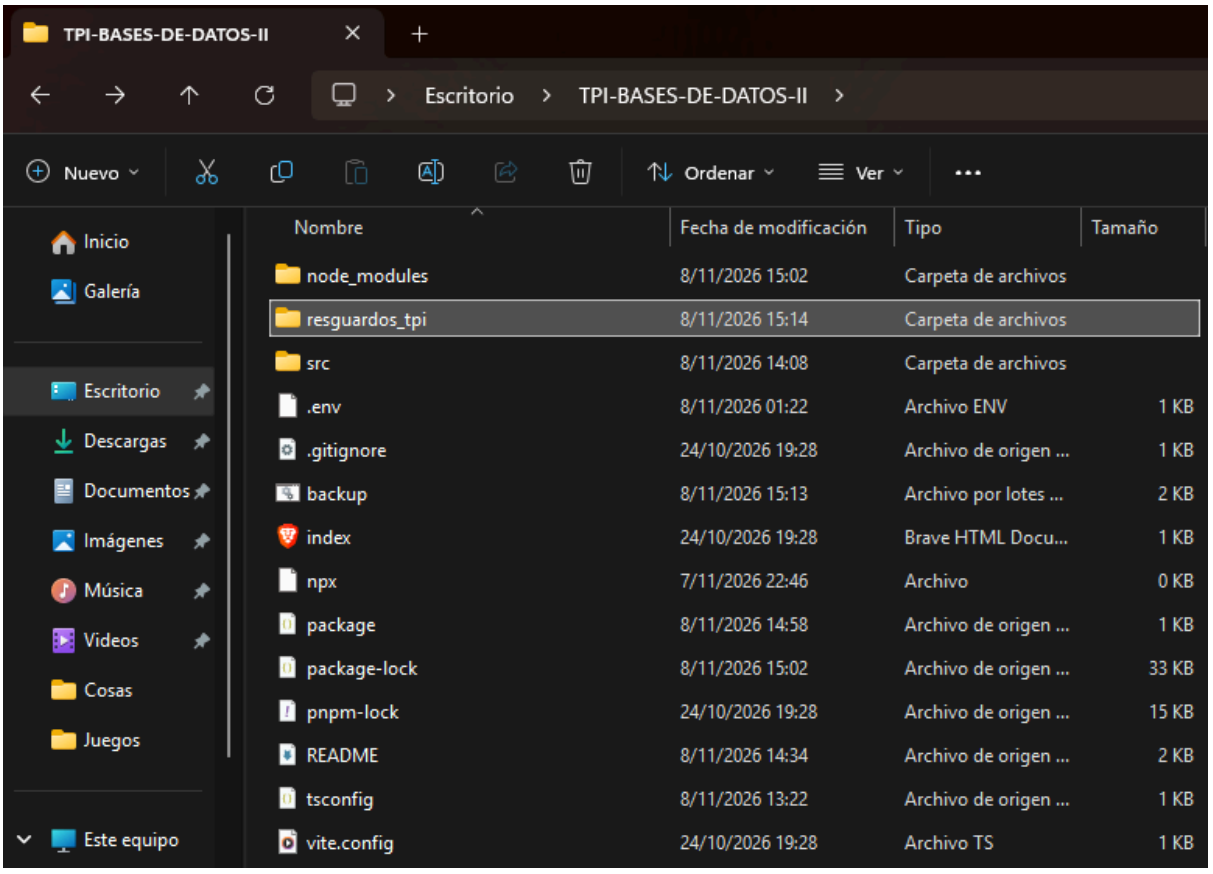
TECNICATURA UNIVERSITARIA
EN PROGRAMACIÓN
A DISTANCIA

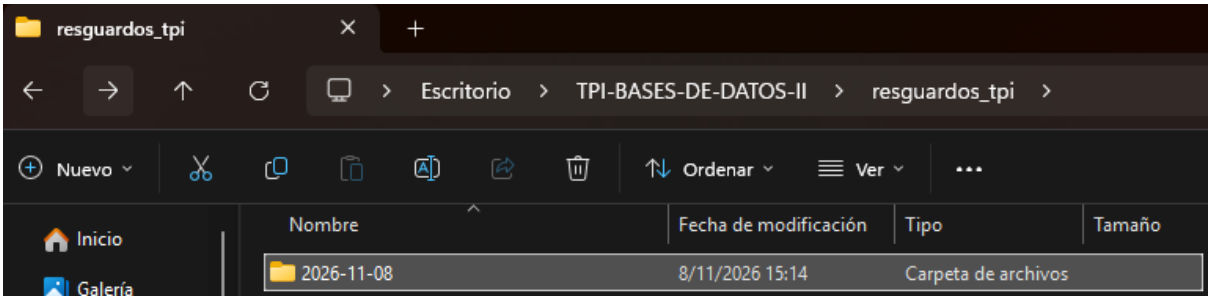





```
C:\Users\santa\OneDrive\Escritorio\TPI-BASES-DE-DATOS-II>npx ts-node src/index.ts
◇ injected env (2) from .env
(node:1628) Warning: Setting the NODE_TLS_REJECT_UNAUTHORIZED environment variable to '0' makes TLS connections and HTTPS requests insecure by disabling certificate
verification.
(Use `node --trace-warnings ...` to show where the warning was created)
 ¡Conexión exitosa a MongoDB Atlas!
 1. CREATE: Usuario insertado con ID: new ObjectId('6af0b9fc6d2383b6bae0f3e7')
 1. CREATE: Producto insertado con ID: new ObjectId('6af0b9fc6d2383b6bae0f3e8')
 1. CREATE (Pedido): Pedido insertado con ID: new ObjectId('6af0b9fd6d2383b6bae0f3e9')
 2. READ: Listado de productos activos encontrados: 1
 3. UPDATE: Producto modificado. Documentos modificados: 1
 4. DELETE (Baja Lógica): Usuario desactivado correctamente: 1
 Conexión cerrada correctamente.
```







6. Conclusión

El desarrollo de este Trabajo Práctico Integrador ha permitido consolidar de manera teórica y práctica los conceptos fundamentales de la arquitectura de bases de datos NoSQL, la persistencia en entornos Cloud distribuidos y la gestión integral del ciclo de vida de los datos. La implementación de un modelo orientado a documentos en MongoDB Atlas demostró ser altamente eficiente para dominios dinámicos como el e-commerce gastronómico, donde la flexibilidad de esquemas facilita la adaptación a requerimientos cambiantes sin sacrificar el rendimiento de las consultas.

Asimismo, el análisis minucioso de los patrones de acceso para justificar la implementación híbrida entre Embedding y Linking evidenció la importancia crítica de diseñar modelos de datos basados en las necesidades reales de lectura y escritura de la aplicación, evitando cuellos de botella y maximizando la escalabilidad horizontal. La incorporación rigurosa de

políticas de baja lógica garantizó la integridad y trazabilidad histórica de la información, cumpliendo con los estándares de auditoría corporativa sin recurrir a eliminaciones físicas destructivas.

Finalmente, la automatización de los respaldos físicos mediante scripts CLI y herramientas nativas como mongodump, complementada con el análisis de métricas de recuperación (RTO y RPO), aportó una perspectiva administrativa indispensable. Se comprendió que una solución de software no solo debe ser funcional a nivel de código y comunicación cliente-servidor, sino también resiliente, segura y preparada para contingencias operativas en entornos de producción reales.